Architecture Review Framework

A reusable documentation and review method, applied to MateMate.

The framework answers three questions during a review. What are the system boundaries? Who can change its important state? What evidence supports the current design? It uses C4 for structural zoom, arc42 for explanations and three required review records for checks that can be repeated after a change.

Required views and sections

Source	Part adopted	Framework deliverable and MateMate location
C4 L1	System context	Scope diagram and the human interaction (p. 3).
C4 L2	Container	One runnable desktop application and its boundary (p. 3).
C4 L3	Component	Seven internal blocks and their directed dependencies (p. 4).
arc42 §1	Introduction and goals	Purpose, scope, stakeholders and quality priorities (pp. 3, 5).
arc42 §2	Constraints	Offline execution, local interaction and chess rules (p. 3).
arc42 §3	Context and scope	External actor, interface and excluded features (p. 3).
arc42 §§4-5	Strategy and building blocks	Threading, responsibility boundaries and interfaces (pp. 3-4).
arc42 §§6-7	Runtime and deployment	Three runtime scenarios and the installation topology (pp. 3, 5).
arc42 §9	Architecture decisions	Recorded decisions with trade-offs and revisit triggers (p. 4).
arc42 §§10-11	Quality, risks and debt	Acceptance scenarios, scorecard and open risks (pp. 5-7).

Where the framework improves the review

Compared with C4, the same structural diagrams are accompanied by explicit decision rationale, state authority and testable quality gates. **Compared with arc42,** reviewers get a fixed zoom sequence, a consistent visual vocabulary and prescribed records for ownership, change boundaries and evidence freshness. Both baselines can accommodate these practices. This framework makes them required deliverables.

This is designed for small systems that need repeatable reviews of correctness and change impact. Its extra records add maintenance work. A pilot can compare the methods on reviewer time to locate a writer, forecast impacted blocks and find current evidence, while checking the answers against the same design.

How to produce and review it

Use stable IDs so a scenario, decision or test result can point to the same element across views.

Review procedure

Step	Action	Output
1. Set scope	Agree on stakeholders, system boundary, constraints and measurable quality scenarios.	Context, goals and Q IDs
2. Decompose	Draw context, containers and relevant components. Record allowed dependencies and describe each interface.	Views, B IDs and edge list
3. Walk through	Trace a normal flow, a failure and one boundary change. Record decisions, state authority and anticipated change impact.	Runtime traces, ADRs, A and P IDs
4. Verify	Bind checks to a baseline and run them at the required maturity. Review missing evidence and metric thresholds.	Evidence register and gate result
5. Maintain	On a change, repeat affected probes and checks. Update views and the evidence baseline before sign-off.	New baseline and refreshed results

Three required additions

Record	Minimum fields and review rule
Authority ledger (A)	State item, sole writer, readers, commit boundary, invariant and rejection behavior. A state item needs exactly one declared authority.
Change-impact probe (P)	Change, required and permitted blocks, protected contracts, touch budget and observed diff. Check both the changed set and the protected contracts.
Evidence and expiry register (E)	Claim, evidence class, check, baseline fingerprint, result and invalidation trigger. Invalidate evidence when a recorded input changes. An open required check blocks its gate.

Diagram notation

Channel	Meaning
Fill color and role	Green A = chess/application logic. Blue T = reusable technology. Gray = person or aggregate boundary. Text also identifies the role.
Frame style	Double = owns committed state in this view. Single = internal element without that authority. Dashed = external participant. The outer dotted outline in a container view encloses the system.
Box size and priority	P1 large = rules, turn or latency critical. P2 standard = supporting. P3 small = optional. Set by review priority within the view. Size does not count classes or effort.
Arrow	Direction is labeled. Structural component arrows mean allowed caller-to-provider dependencies. Context and container arrows show interactions.

MateMate · Context and deployment

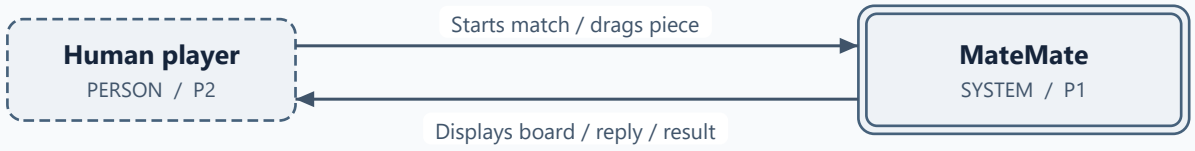
Scope, stakeholders and C4 levels 1 and 2.

Goals and constraints

A human plays the computer locally through a drag-and-drop chessboard. The player needs legal moves and a responsive reply. Maintainers need to change rules, search and the GUI independently. A course reviewer needs to trace key moves and inspect design evidence.

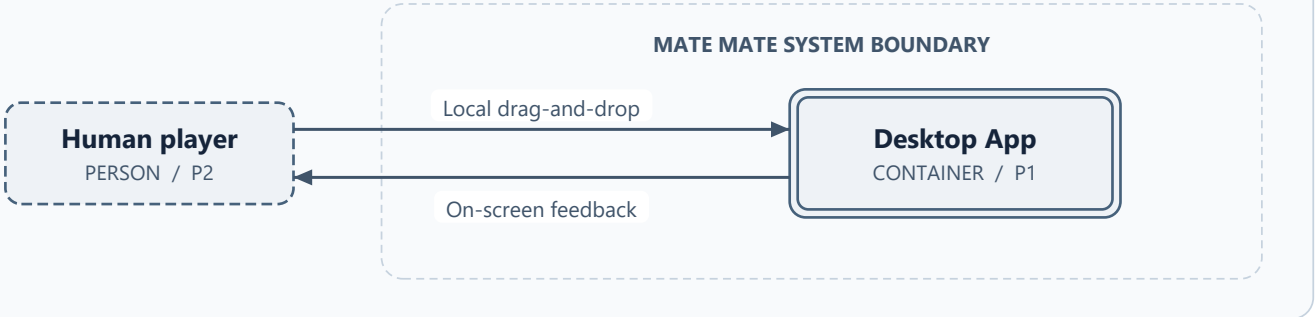
The design assumes one active match and standard board orientation controls. The match lives in memory. Online play, saved games, tournament clocks and an explicit threefold/fifty-move claim interface are outside this iteration. Rules include automatic fivefold and seventy-five-move draws with checkmate precedence, and distinguish those from player-claimable draws.

C4 level 1 · System context



The human is the only external actor. The computer opponent is part of MateMate. The local interface accepts start and move actions and presents board state, invalid-move feedback and game results.

C4 level 2 · Containers

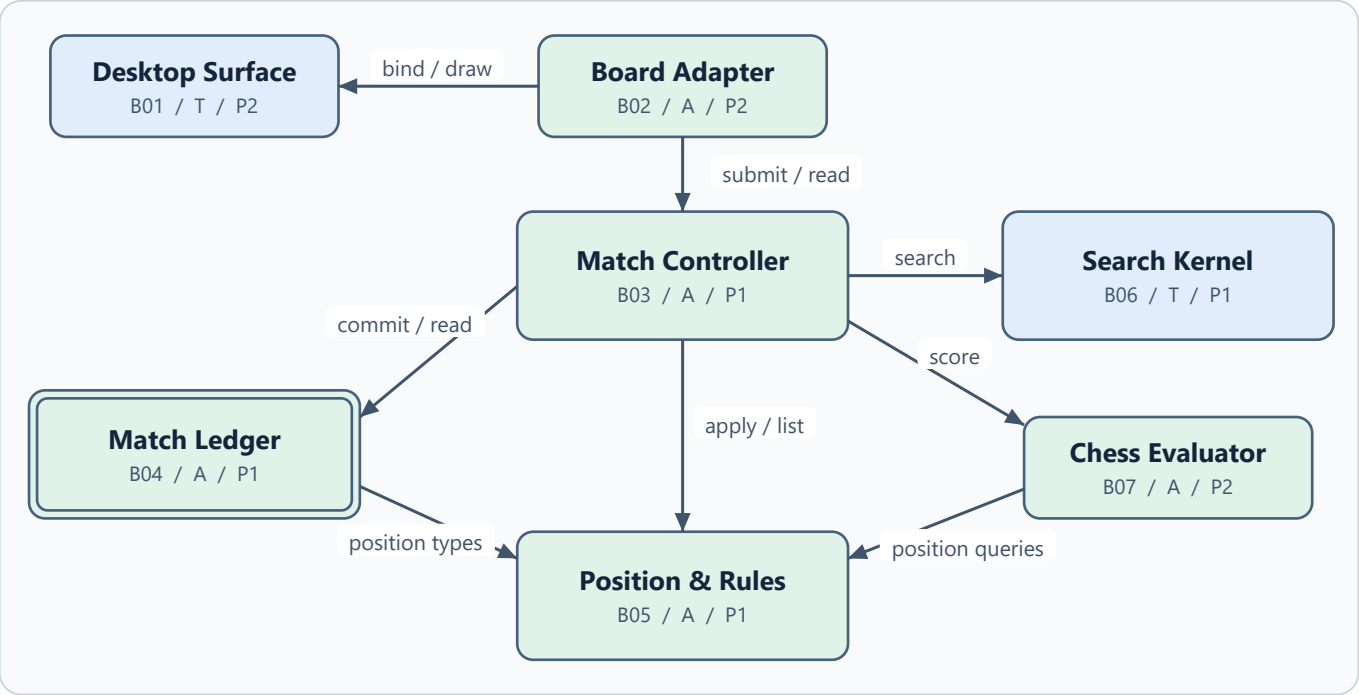


The proposed desktop app is one Java process with a JavaFX shell, an in-memory game ledger and an in-process search worker. Deploy a bundled desktop app and compatible runtime on the player's workstation. There is no inter-process protocol or server deployment. The GUI and worker communicate through in-process immutable messages.

Key. Gray marks the player or system/container aggregate. Dashed marks the external player. Double marks the state-owning aggregate. P1 is critical to the system and P2 is supporting. Arrows carry the labeled interactions.

MateMate · Components and decisions

C4 level 3 inside the Desktop App. All components are in-process Java modules. B01 is the JavaFX adapter.



Key. Green A = chess logic. Blue T = technology. Double = committed-state owner. Large P1 = review-critical. Standard P2 = supporting. Each arrow gives an allowed caller-to-provider dependency.

Contracts. B02 calls B03 with a square-based intent and renders immutable view data. B03 gives B06 a generic snapshot and legal-transition, terminal and evaluation callbacks. B05 owns rule logic. B04 is the only commit API. All eight allowed edges are shown above. Unlisted edges are forbidden, and reusable T blocks can only call their generic contracts.

Architecture decisions

Decision	Choice and trade-off	Revisit when
ADR-01 **Position and history**	Use immutable positions and a separate match ledger. In-place apply/undo saves allocations, while copies simplify branching, rejection and history replay.	A representative search benchmark misses Q3 and profiling attributes the cost to position allocation.
ADR-02 **Search interface**	Use a generic bounded minimax kernel and a separate chess evaluator. This adds an adapter and allows the evaluator or search to be changed separately.	The adapter becomes a measured bottleneck or the engine requires a different model of search.
ADR-03 **Commit model**	Use one serialized controller and a single-writer ledger. A background worker may compute candidates. A token/revision check fences off stale results.	A second local or remote writer becomes a requirement.

MateMate · Runtime and quality

Three runtime scenarios connect the architecture views to release acceptance checks.

Runtime scenarios

Scenario	Cooperation and data
R1 · Human move and reply	B01 sends the drop to B02, which maps (824, 535) to e4 using the White-oriented board origin (500, 175) and 80-pixel cells. B02 sends e2-e4 and the match revision to B03. B05 applies a legal move and B04 commits. B03 schedules B06 and binds B05 transitions and B07 scores. B03 validates and commits the returned reply, then B02 redraws the snapshot.
R2 · Castle	B03 sends the requested king move to B05. B05 checks WK, WQ, BK or BQ, the home pieces, empty path and safe king squares. Returning a king or rook never restores a cleared right. A valid castle changes both pieces in one transition and B04 appends one move. Rejection leaves the position and revision intact.
R3 · Timeout or stale result	B03 gives B06 a 750 ms cooperative budget and fences the result with match ID, revision and token. B03 uses a completed result or a prevalidated fallback on deadline. Restart or the first accepted reply invalidates the token. Later messages cannot commit. The GUI continues processing events during search.

Quality scenarios and acceptance

ID	Stimulus and measurement	Pass condition
Q1 Correctness	Run 12 castling fixtures: four legal castles, four king/rook-return cases across both colors, two captured-rook replacements and two blocked/attacked paths. Compare snapshot, rights and history before and after.	All 12 outcomes match. Valid cases add one move and rejected cases add zero.
Q2 Integrity	Inject a stale reply after restart, a duplicate reply and a reply with an incorrect revision. Compare ledger revision and accepted moves.	All three injected replies cause zero commits.
Q3 Responsiveness	Warm the release build, then run 100 fixed position-and-move pairs that leave a nonterminal engine turn, at standard difficulty on a recorded 4-core/16 GB workstation. Measure accepted drop to the first GUI paint containing the legal reply.	Nearest-rank p95 at most 1,000 ms. Every run yields exactly one legal reply.
Q4 Changeability	Execute probes P1 and P2 with one change per baseline. Compare changed production blocks and protected contracts.	Both probes stay within their permitted block sets and budgets and preserve the listed contracts.

Q1 and Q2 gate correctness. Q3 and Q4 gate the agreed latency and maintainability targets. These are proposed acceptance criteria for an implementation.

MateMate · Review records

The proposed design is baseline D1. Structural findings below refer to the documented model.

A · Authority ledger

State item	Sole writer and readers	Commit rule
A1 · Committed board, rights and revision	B04 writes. B03 reads directly, B02 and B06 receive snapshots through B03.	Append a legal transition only at the expected match ID and revision. Reject without mutation.
A2 · Accepted history and repetition counts	B04 writes. B03 reads and seeds B06 with a copy.	Update history and repetition counts with A1. Failed moves append nothing.
A3 · Active turn phase and search token	B03 writes. B02 reads the phase in a view snapshot.	Update on the serialized command loop. Reject results for a retired token or mismatched revision.

P · Change-impact probes

Probe	Required and permitted production blocks	Protected contracts and status
P1 · Revise the positional heuristic	{B07} with a budget of one block.	Preserve the evaluator interface and B05 position/rules contract. Predicted {B07}.
P2 · Replace the desktop toolkit	{B01} with a budget of one block. Platform packaging/config may change separately and is recorded.	Preserve B01 pointer/drawing contracts and B02 square mapping. Predicted {B01}.

E · Evidence and expiry register

ID / claim	Check and baseline	Status and expiry trigger
E0 · Graph and authority	Design inspection of D1: seven blocks, eight arrows and A1-A3.	Reviewed at design level. Expires when the block list, edge list or A rows change.
E1 / Q1 · Castle	Fixtures for R2 on a named build and fixture revision.	Pending. Repeat after rules, history or fixture changes.
E2 / Q2 · Stale reply	Three injected cases for R3 on a named build and fixture revision.	Pending. Repeat after scheduling, commit or fixture changes.
E3 / Q3 · Latency	Fixed workload for R1 on a named build, settings and machine.	Pending. Repeat after a relevant build, workload, settings or machine change.
E4 / Q4 · Change	P1 and P2 diffs and contract checks, each pinned to before/after revisions.	Pending. Repeat after a protected interface or probe definition changes.

Record the immutable build revision, test or probe revision and relevant configuration for each implementation check. An expired result is excluded until rerun.

MateMate · Architecture scorecard

Compute each indicator against a named baseline. Keep the indicators separate so structural strengths cannot mask failed correctness checks.

Metric	Calculation and release target	MateMate at D1
Dependency conformance	Allowed observed edges ÷ all observed direct edges. Target 1.00. Review the design graph and compare implementation module imports before release.	8/8 = 1.00 on the modeled graph.
Cycle freedom	1 - (blocks in a directed cycle ÷ all blocks). A self-loop counts. Target 1.00. Use strongly connected components to count affected blocks once.	1 - 0/7 = 1.00 on the modeled graph.
State authority coverage	Critical state items with exactly one declared writer ÷ all inventoried critical state items. Target 1.00. Validate real write paths at implementation review.	3/3 = 1.00 in the design ledger.
Change isolation	Probes within permitted blocks and budgets that also touch the required blocks and preserve contracts ÷ executed probes. Target 1.00 with both probes executed.	Pending. Predicted impact is one block per probe.
Current acceptance evidence	Required quality checks with current passing implementation evidence ÷ required checks. Target 1.00. Q1-Q4 are each one check group.	0/4 = 0.00 evidence coverage. Q1-Q4 are pending.
Timely legal replies	Cases producing one legal painted reply within 1,000 ms ÷ 100 reference cases. Target ≥ 0.95 and all replies legal. Equivalent to the latency bound in Q3 for 100 successful runs.	Pending. Record p95 latency in ms alongside the ratio.

A structural metric with an empty denominator is marked not applicable, and an unexecuted probe has no observed isolation score. Release remains blocked until Q1-Q4 each have current passing evidence.

Risks and planned checks

Search cost. Immutable positions may create allocation pressure, and tight budgets may reduce playing strength. Profile E3 and retain search depth and fallback frequency. **Rule-state drift.** An incorrect castling flag or repetition key can affect both live and hypothetical positions. Share B05 transitions and add repetition fixtures when implementing the automatic draw rules.

Sources

Martin Hasitschka, *Architecture Documentation*, V704, PDF pp. 5, 9-10 and 72.

Martin Hasitschka, *Architecture Development*, V702, PDF pp. 27-36 (MateMate and SE4).

C4 model diagrams and notation.

arc42 documentation, sections 1-12.

FIDE Laws of Chess, arts. 3.8-3.9 and 9.2-9.6.